

Construction Company Functions

Technical programming document: Oracle database, Python backend, and Tkinter frontend

1. System Overview

The project implements a desktop application for a construction company. The database runs on Oracle XE inside the XEPDB1 PDB, and the final schema is ConstructionCompany. The application is separated into a Tkinter frontend in construction_manager_app.py and a Python backend inside scripts/, where each file groups queries or processes related to one table or functional area.

Item	Technical detail
Database	Oracle XE, Service Name XEPDB1, host localhost, port 1521.
User / schema	ConstructionCompany.
Credentials used by the scripts	USER=ConstructionCompany, PASSWORD=const123, dsn=localhost:1521/XEPDB1.
Valid DDL	database/Constructora_fixed.ddl. The original ERwin DDL is not used directly because it generated RI triggers and incorrect data types.
Seed data	database/Constructora_seed_data.sql with records for all tables and 40 test payments for weeks 1 to 4 of 2026.
Execution	python construction_manager_app.py or double-click run_construction_manager.bat.

2. Project Structure

- construction_manager_app.py: Tkinter frontend, login, navigation, forms, asynchronous execution, and results.
- run_construction_manager.bat: Windows launcher; detects py or python and keeps the window open if errors occur.
- scripts/: Python backend with Oracle queries by functional module.
- scripts/ReferenceData.py: automatic helper lists loaded when entering each module.
- database/Constructora_fixed.ddl: final schema definition with declarative PK, FK, and CHECK constraints.
- database/Constructora_seed_data.sql: initial data and test payments.
- assets/duo_builders_logo.png: logo used by the window, login screen, and header.

3. Oracle Data Model

The model keeps primary keys, foreign keys, and CHECK restrictions as declarative constraints. ERwin-generated triggers were removed, and incorrect data types were corrected so the DDL is executable and consistent with the application.

Customers

Columns	Primary key	Foreign keys	Notes
Customer_ID, Name, Phone, Email, CustomerType, RegistrationDate	Customer_ID	N/A	Stores the construction company's customers. Name is VARCHAR2(80).

ConstructionSite

Columns	Primary key	Foreign keys	Notes
ConstructionSite_ID, Customer_ID, Country, City, Street, Address	ConstructionSite_ID	Customer_ID -> Customers	Stores construction sites optionally linked to a customer. Uses ON DELETE SET NULL.

Employees

Columns	Primary key	Foreign keys	Notes
Employee_ID, Boss_ID, ConstructionSite_ID, FirstName, LastName, StartDate, HireDate, CurrentPay, BankAccount, BankName, Phone	Employee_ID	Boss_ID -> Employees; ConstructionSite_ID -> ConstructionSite	Includes a self-reference for managers and assignment to a construction site.

EmployeePayment

Columns	Primary key	Foreign keys	Notes
WeekNumber, Year, Employee_ID, PaymentAmount, Tax, PayDate	WeekNumber, Year, Employee_ID	Employee_ID -> Employees	Registers payments by employee, year, and week. WeekNumber has CHECK 1-53.

Machines

Columns	Primary key	Foreign keys	Notes
Machine_ID, Model, Type, IsUsed	Machine_ID	N/A	IsUsed is NUMBER(1) with CHECK IN (0,1).

Licenses

Columns	Primary key	Foreign keys	Notes
Employee_ID, Machine_ID, LicenseType, InitialLicenseDate, EndLicenseDate	Employee_ID, Machine_ID	Employee_ID -> Employees; Machine_ID -> Machines	N:M relationship between employees and machines.

MachinesInConstruction

Columns	Primary key	Foreign keys	Notes
ConstructionSite_ID, Machine_ID, StartMachineDate, FinishMachineDate	ConstructionSite_ID, Machine_ID	ConstructionSite_ID -> ConstructionSite; Machine_ID -> Machines	N:M relationship between construction sites and machines.

WareHouse

Columns	Primary key	Foreign keys	Notes
WareHouse_ID, ConstructionSite_ID, Name, Country, City, Street, Address, Phone	WareHouse_ID	ConstructionSite_ID -> ConstructionSite	Warehouses assigned to construction sites. Phone is stored as VARCHAR2(20).

Material

Columns	Primary key	Foreign keys	Notes
Material_ID, Name, Purpose	Material_ID	N/A	Purpose stores the material usage.

Stock

Columns	Primary key	Foreign keys	Notes
WareHouse_ID, Material_ID, QuantityKg	WareHouse_ID, Material_ID	WareHouse_ID -> WareHouse; Material_ID -> Material	Stock by warehouse/material. QuantityKg >= 0.

ServiceExternalCompany

Columns	Primary key	Foreign keys	Notes
---------	-------------	--------------	-------

Company_ID, CompanyName, Phone, Email	Company_ID	N/A	Catalog of external service providers.
---------------------------------------	------------	-----	--

ServiceInConstruction

Columns	Primary key	Foreign keys	Notes
ConstructionSite_ID, Company_ID, ServiceDescription, TypeService, InitDateService, EndDateService	ConstructionSite_ID, Company_ID	ConstructionSite_ID -> ConstructionSite; Company_ID -> ServiceExternalCompany	N:M relationship between construction sites and external companies.

4. Main Relationships

- Customers 1:N ConstructionSite.
- ConstructionSite 1:N Employees.
- Employees 1:N EmployeePayment.
- Employees has a self-relationship through Boss_ID.
- Employees N:M Machines through Licenses.
- ConstructionSite N:M Machines through MachinesInConstruction.
- ConstructionSite 1:N WareHouse.
- WareHouse N:M Material through Stock.
- ConstructionSite N:M ServiceExternalCompany through ServiceInConstruction.

5. Backend Python

Each script opens an oracledb connection using the same dsn, creates a cursor, executes a parameterized query, reads results with fetchone() or fetchall(), closes the cursor/connection, and returns text ready to be displayed in the frontend. When no records are found, the common value failVariable = 'No variable found in the DB' is returned.

- fetchone(): used when only one record is expected.
- fetchall(): used when a query can return multiple rows; each row is received as a tuple.
- Oracle parameters: bind variables such as :employee_id are used to avoid direct SQL string concatenation.
- Results are converted to strings because the frontend displays plain text in the Results panel.

scripts/ConstructionSite.py

Function	Parameters	Operation	Main query / rule	Return value
ConstructionInfo	Construction_ID	Retrieves location and customer data for a construction site.	LEFT JOIN ConstructionSite with Customers by Customer_ID.	String with country, city, street, address, and customer; returns failVariable if no data is found.
SearchConstructionSiteByName	Name	Searches construction sites by customer name.	JOIN ConstructionSite with Customers and filters with upper(c.name) LIKE upper(:customer_name).	List of matching construction sites, one per line.
SearchConstructionSiteByID	ConstructionSite_ID	Searches for one construction site by ID.	LEFT JOIN with Customers and filter by ConstructionSite_ID.	String with the full construction site information.
AllOfTheEmployeesInTheConstruction	ConstructionSite_ID	Lists employees assigned to a construction site.	SELECT from Employees filtered by ConstructionSite_ID.	Lines with Employee_ID, FirstName, and LastName.

scripts/Customers.py

Function	Parameters	Operation	Main query / rule	Return value
----------	------------	-----------	-------------------	--------------

CustomerInfo	Customer_ID	Retrieves general customer information.	SELECT Name, Phone, Email, and CustomerType from Customers.	String with name, phone, email, and customer type.
SearchCustomerTypeByID	Customer_ID	Retrieves only the customer type.	SELECT CustomerType from Customers.	CustomerType or failVariable. It still exists in the backend, but it is no longer shown in the frontend.

scripts/Employees.py

Function	Parameters	Operation	Main query / rule	Return value
ListEmployeesForPayment	No parameters	Lists employees available for payroll.	SELECT Employee_ID and full name from Employees.	List in 'Employee ID Name' format. This was the first helper list for Payroll.
ConsultEmployeesUnderManagers	No parameters	Links each employee with their manager.	Self join Employees e JOIN Employees b ON e.Boss_ID = b.Employee_ID.	Lines with employee and manager.
EmployeeInfo	Employee_ID	Retrieves employment and contact data for an employee.	SELECT full name, StartDate, HireDate, and Phone from Employees.	String with name, dates, and phone.
EmployeesInBank	BankName	Searches employees that use a specific bank.	SELECT full name and BankAccount from Employees WHERE BankName = :bank_name.	Lines with employee and bank account.

scripts/EmployeePayment.py

Function	Parameters	Operation	Main query / rule	Return value
_format_money	value	Formats monetary values.	Converts to float and applies two decimal places.	Text in 0.00 format.
_last_tax_for_employee	cursor, Employee_ID	Gets the employee's most recent historical tax value.	Searches non-null Tax in EmployeePayment ordered by Year and WeekNumber descending.	Tax as float; 0.0 if there is no history.
_employee_payment_data	cursor, Employee_ID	Gets base data required to calculate a payment.	SELECT Employee_ID, full name, and CurrentPay from Employees.	Tuple with employee data or None.
ListEmployeesToPay	Year, WeekNumber	Lists every employee and payment status for one week.	LEFT JOIN Employees with EmployeePayment by employee, year, and week.	Lines with CurrentPay, PaymentAmount, Tax, NetPayment, and Paid/Pending status.
PayEmployee	Year, WeekNumber, Employee_ID	Registers the weekly payment for an employee.	Validates employee; checks whether payment already exists; inserts into EmployeePayment if it does not exist.	Message for registered payment or already existing payment.
ConsultPaidPeriod	Year, WeekNumber, Employee_ID	Consults the payment amount for one week.	SELECT PaymentAmount in EmployeePayment by composite PK.	PaymentAmount as text or failVariable.
ConsultDeductibleTax	Year, WeekNumber, Employee_ID	Consults the deducted tax.	SELECT Tax in EmployeePayment by composite PK.	Tax as text or failVariable.

PayWeek	Year, WeekNumber, Employee_ID	Checks whether a week was paid.	Counts records with non-null PayDate for employee/year/week.	'True' if a payment exists; 'False' otherwise.
---------	-------------------------------------	------------------------------------	--	--

scripts/Licenses.py

Function	Parameters	Operation	Main query / rule	Return value
EmployeesWithSameLicense	LicenseType	Lists employees that share the same license type.	JOIN Licenses with Employees and filter by LicenseType.	Lines with employee, machine, and license type.

scripts/Machines.py

Function	Parameters	Operation	Main query / rule	Return value
ListAllMachines	No parameters	Lists every machine.	SELECT Machine_ID, Model, Type, IsUsed from Machines.	Lines with machine data.

scripts/MachinesInConstruction.py

Function	Parameters	Operation	Main query / rule	Return value
ListAllMachinesInConstruction	ConstructionSite_ID	Lists machines assigned to a construction site.	JOIN MachinesInConstruction with Machines by Machine_ID.	Lines with machine, model, type, and usage dates.

scripts/Material.py

Function	Parameters	Operation	Main query / rule	Return value
MaterialPurpose	Name	Consults the purpose of a material.	SELECT Purpose from Material with case-insensitive Name comparison.	Purpose or failVariable.

scripts/ServiceExternalCompany.py

Function	Parameters	Operation	Main query / rule	Return value
ServiceExternalCompanies	No parameters	Lists external companies.	SELECT CompanyName, Phone, and Email from ServiceExternalCompany.	Lines with contact data.

scripts/ServiceInConstruction.py

Function	Parameters	Operation	Main query / rule	Return value
ListServiceCompanyByTypeService	TypeService	Searches companies by provided service type.	JOIN ServiceInConstruction with ServiceExternalCompany and filter by TypeService.	Lines with company, contact, construction site, and service description.

scripts/Stock.py

Function	Parameters	Operation	Main query / rule	Return value
CurrentStockQuantity	WareHouse_ID, Material_ID	Consults the current material quantity in a warehouse.	SELECT QuantityKg from Stock by WareHouse_ID and Material_ID.	QuantityKg as text or failVariable.

scripts/WareHouse.py

Function	Parameters	Operation	Main query / rule	Return value
ListAllWarehouses	No parameters	Lists all warehouses.	SELECT Name, Country, City, Street, Address, and Phone from WareHouse.	Lines with full warehouse data.
WhichWareHousesAreInTheConstructionSite	ConstructionSite_ID	Lists warehouses assigned to a construction site.	SELECT from WareHouse filtered by ConstructionSite_ID.	Lines with WareHouse_ID and warehouse data.

scripts/ReferenceData.py

Function	Parameters	Operation	Main query / rule	Return value
_run_reference_query	title, query	Executes a simple reference query.	Opens a connection, executes the query, and formats two columns as 'ID Name'.	Text with title and reference rows.
_run_combined_reference_query	parts	Executes several reference queries in one output.	Reuses one connection and iterates through title/query pairs.	Text grouped by sections.
ListConstructionSiteReferences	No parameters	Shows construction site IDs with customer and city.	LEFT JOIN ConstructionSite with Customers.	Reference list for the Construction Sites module.
ListCustomerReferences	No parameters	Shows customer IDs and names.	SELECT Customer_ID, Name from Customers.	Reference list for Customers.
ListEmployeeReferences	No parameters	Shows employees and licenses.	Combines Employees and Licenses with Employees.	Reference list for Employees.
ListPaymentReferences	No parameters	Shows existing payments and employees.	Combines EmployeePayment with Employees and lists Employees.	Reference list for Payments.
ListPayrollPaymentReferences	No parameters	Shows employees available for payroll.	SELECT Employee_ID and name from Employees.	Reference list for Payroll Payment.
ListMachineReferences	No parameters	Shows machines and machines by construction site.	Combines Machines and MachinesInConstruction with Machines.	Reference list for Machines.
ListMaterialWarehouseReferences	No parameters	Shows materials, warehouses, and stock.	Combines Material, WareHouse, and Stock with joins.	Reference list for Materials & Warehouses.
ListExternalServiceReferences	No parameters	Shows external companies and services by construction site.	Combines ServiceExternalCompany and ServiceInConstruction.	Reference list for External Services.

6. Tkinter Frontend

The frontend is implemented in `construction_manager_app.py` with a main `ConstructionManagerApp` class that inherits from `tk.Tk`. The interface includes login, a logo header, module sidebar, operation dropdown, dynamic form, results panel, and status line. Operations are configured declaratively with the `Field` and `Operation` dataclasses.

Method / element	Parameters or data	Responsibility	Technical notes
Field	key, label, kind='text'	Dataclass that defines a form field.	kind='int' enables numeric conversion and validation.
Operation	title, description, module, function, fields	Dataclass that links a frontend option to a backend function.	module points to a file inside <code>scripts/</code> .
SECTIONS	dict[str, tuple[Operation]]	Main map for the sidebar and operation dropdown.	Controls the options visible to the user.
SECTION_REFERENCE_OPERATIONS	dict[str, Operation]	Map for automatic helper lists by module.	Uses <code>ReferenceData.py</code> and in-memory cache.
__init__	self	Initializes the window, state, cache, queue, and login screen.	Defines <code>current_section</code> , <code>current_operation</code> , and module structures.
_configure_style	self	Configures ttk theme, colors, fonts, and buttons.	Uses the clam theme and Duo Builders palette.
_set_window_icon	self	Loads the window icon from <code>assets/duo_builders_logo.png</code> .	Returns without failure if the file does not exist.
_build_logo	parent, subsample, background	Builds the logo label for login/header.	Stores references in <code>self.logo_images</code> to avoid garbage collection.
_build_login	self	Builds the authentication screen.	Expected user: <code>ConstructionCompany</code> ; password: <code>const123</code> .
_attempt_login	self	Validates local credentials and opens the app.	On failure, clears the password and shows Invalid user or password.
_build_layout	self	Builds the header, sidebar, query panel, results panel, and status line.	Selects <code>Construction Sites</code> as the initial module.
_build_query_panel	parent	Builds the query dropdown, description, dynamic form, and buttons.	Run Query executes the current operation.
_build_results_panel	parent	Builds the results area with scroll, Copy, and Clear.	The Text widget uses <code>Consolas</code> for tabular output.
_on_section_selected	event	Handles sidebar module changes.	Repopulates operations and shows the first option in the module.
_populate_operations	section	Loads module operations into the combobox.	Updates <code>operation_lookup</code> and <code>current_section</code> .
_show_operation	title	Shows the description and required fields for an operation.	If there are no fields, it still loads the reference list.
_load_section_reference	self	Loads the module's automatic helper list.	Uses <code>reference_cache</code> and <code>loading_reference_sections</code> to avoid repeated queries.
_clear_fields	self	Clears the current form inputs.	Does not clear results.

_run_current_operation	self	Validates arguments and starts the query in a background thread.	Disables Run Query while execution is in progress.
_collect_args	self	Collects and validates input values.	Empty fields show a warning; kind='int' requires a whole number.
_execute_operation	operation, args	Imports the backend module and executes the function with arguments.	Sends the result to output_queue.
_execute_reference_operation	section, operation	Executes the module reference function.	Marks output as a reference for cache and current-section control.
_format_query_error	exc	Simplifies errors for the end user.	ORA-/DPY-/DPI-/TNS errors show Connection failed.; all others show Query failed.
_load_module	filename	Dynamically loads a Python script from scripts/.	Caches modules in self.modules with importlib.util.
_poll_output_queue	self	Checks the background thread result queue.	Prevents an old reference list from being displayed after the user changes modules.
_set_results	value	Writes text to the Results panel.	Leaves the Text widget in normal state so the user can select/copy content.
_copy_results	self	Copies the Results content to the clipboard.	Updates status to Results copied.

7. Available Interface Operations

Module	Visible option	Script	Backend function	Inputs
Construction Sites	Construction Info	ConstructionSite.py	ConstructionInfo	construction_id
Construction Sites	Search Construction By Customer	ConstructionSite.py	SearchConstructionSiteByName	name
Construction Sites	Search Construction By ID	ConstructionSite.py	SearchConstructionSiteByID	construction_site_id
Construction Sites	Employees In Construction	ConstructionSite.py	AllOfTheEmployeesInTheConstruction	construction_site_id
Customers	Customer Info	Customers.py	CustomerInfo	customer_id
Employees	Employees Under Managers	Employees.py	ConsultEmployeesUnderManagers	No parameters
Employees	Employee Info	Employees.py	EmployeeInfo	employee_id
Employees	Employees By Bank	Employees.py	EmployeesInBank	bank_name
Employees	Employees By License	Licenses.py	EmployeesWithSameLicense	license_type
Payments	Paid Amount	EmployeePayment.py	ConsultPaidPeriod	year:int, week_number:int, employee_id

Payments	Deductible Tax	EmployeePayment.py	ConsultDeductibleTax	year:int, week_number:int, employee_id
Payments	Week Was Paid	EmployeePayment.py	PayWeek	year:int, week_number:int, employee_id
Payroll Payment	Employees To Pay	EmployeePayment.py	ListEmployeesToPay	year:int, week_number:int
Payroll Payment	Register Employee Payment	EmployeePayment.py	PayEmployee	year:int, week_number:int, employee_id
Machines	All Machines	Machines.py	ListAllMachines	No parameters
Machines	Machines In Construction	MachinesInConstruction.py	ListAllMachinesInConstruction	construction_site_id
Materials & Warehouses	Material Purpose	Material.py	MaterialPurpose	name
Materials & Warehouses	Current Stock Quantity	Stock.py	CurrentStockQuantity	warehouse_id, material_id
Materials & Warehouses	All Warehouses	WareHouse.py	ListAllWarehouses	No parameters
Materials & Warehouses	Warehouses In Construction	WareHouse.py	WhichWareHousesAreInTheConstructionSite	construction_site_id
External Services	External Companies	ServiceExternalCompany.py	ServiceExternalCompanies	No parameters
External Services	Companies By Service Type	ServiceInConstruction.py	ListServiceCompanyByTypeService	type_service

8. Flujo de Payroll Payment

Payroll Payment is a separate module from the historical Payments module. Its purpose is to prepare and register real EmployeePayment records from the app.

- Employees To Pay receives Year and WeekNumber; it lists all employees with CurrentPay, PaymentAmount, Tax, NetPayment, and Paid/Pending status.
- Register Employee Payment receives Year, WeekNumber, and Employee_ID; it validates whether the employee exists and whether that week has already been paid.
- The calculation rule is $\text{PaymentAmount} = \text{CurrentPay} / 4$.
- Tax is taken from the latest historical Tax value registered for the employee; if none exists, 0.0 is used.
- PayDate is inserted with TRUNC(SYSDATE).
- If a record already exists for Year, WeekNumber, and Employee_ID, the function does not insert duplicates and returns 'This employee has already been paid.'

9. Automatic Reference Lists

When entering a module, the frontend executes a ReferenceData.py function to show useful IDs and descriptive data before running a query. These lists are stored in self.reference_cache, and concurrent loading is protected with self.loading_reference_sections.

- Each list is queried only once per module during the session.
- If the user changes modules while a list is loading, the result is cached but does not overwrite the current screen.
- The references help users capture valid IDs for customers, employees, construction sites, machines, materials, warehouses, payments, and services.

10. Error Handling and Validation

- Local login: only allows User=ConstructionCompany and Password=const123.
- Required fields: _collect_args prevents execution when a value is missing.

- Numeric fields: Field.kind='int' converts Year and WeekNumber to integers and shows a warning if they are not whole numbers.
- Oracle/TNS/DPY/DPI errors: displayed as 'Connection failed.' so technical details are not exposed to the end user.
- Other errors: displayed as 'Query failed.'.
- The Run Query button is disabled during execution to avoid duplicate queries.

11. Execution, Installation, and Distribution

- Run SQL scripts in SQL Developer connected as ConstructionCompany with F5.
- If the Python Oracle package is missing, install it with pip install oracledb.
- To open the app: python construction_manager_app.py.
- To open it from Windows: run_construction_manager.bat.
- To generate an executable: pyinstaller --onefile --windowed construction_manager_app.py.
- If the app is distributed, rebuild the .exe after changes in construction_manager_app.py or scripts/.

12. Completed Verifications

- Python script syntax was validated.
- The new ReferenceData.py functions were tested against Oracle.
- ListCustomerReferences() was tested and returned customers correctly.
- PayEmployee(2026, 5, 'E001') was tested: it first registered the payment and then detected the duplicate.
- Constructora_seed_data.sql was updated so a new database starts with 40 payment records.